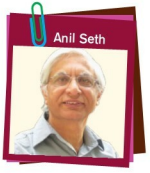

Using Gestures with Touchscreens on KDE5

Using the touchscreen on the netbook can be disappointing, while gestures also fail to please. In this article, the author recalls a personal of using a touchscreen and gestures on a notebook running KDE5.

When I decided to try the touchscreen on the netbook, I was in for a disappointment. I tried scrolling the page in Firefox using the touchpad two-finger pan gesture, and nothing happened! I tried various applications in the Fedora 21 KDE4 environment and it appeared that the touchscreen worked only as a single button mouse. The Qt5 framework supports multi-touch gestures. Hence, the touchscreen experience on a Fedora 22 KDE5 desktop was far better. The window-handling worked very nicely. However, the touchscreen experience within an application depended on the application, e.g., the pan gesture worked on Google Chrome but not on Firefox or Konqueror.

I tried Ubuntu 15.04 and the experience was better. Scrolling using the touchscreen worked in Ubuntu's default browser. I tried pinch-to-zoom but that did not work though it did on Chromium. Moving windows around was nice and easy. However, the touchscreen experience within each application varied a lot.

The article, *https://lwn.net/Articles/485484/*, helped me understand the state of multi-touch gestures in Linux. Xorg is committed to backward compatibility; hence, there is a reluctance to accept gestures as a part of the framework when the gestures are not standardised yet. Instead, Xinput will send touch events to the applications, and each application will have to decide what to do with them.

Routing of touch events and gestures is complicated for desktop managers because of the multiple windows for the different applications. On typical touchscreen devices like the phone and tablet, there's only one application window; thus there is no ambiguity about the destination for a gesture. Hence, the delayed support for touch on desktops and laptops is reasonable, especially as most do not have touchscreens.

KDE5 and Gnome3 have support for recognising some touch gestures; however, each application program needs to accept a gesture and write the code for its functionality.

In this article, we explore touch gestures using Qt5.

The pan gesture

The pan gesture is built into the scrollable widgets in Qt5. Hence, if an application is converted to Qt5, the scrolling motion, as on a touch pad, should work on the touchscreen as well. Shown below is an example of a minimal notepad:

```
#!/usr/bin/python
import sys
from PyQt5.QtWidgets import QWidget, QTextEdit, QApplication
class Notepad(QWidget):
    def __init__(self):
        super(Notepad,self).__init__()
        note = QTextEdit(self)
        note.resize(250,200)
        self.setWindowTitle('Notepad')

if __name__ == '__main__':
    app = QApplication(sys.argv)
    note = Notepad()
    note.show()
    sys.exit(app.exec_())
```

This simple example has only one text window. You can enter enough text so that a scrollbar shows up. You can then use the two-finger pan gesture on the touchscreen to verify that scrolling works. Hence, once the applications migrate to KDE 5/Qt5, scrolling on touchscreens should work on all KDE applications with scrollable widgets.

The pinch gesture

The simplest example of using the pinch gesture to scale is an image viewer. Start with an image viewer sample of PyQt5 *(https://github.com/baoboa/pyqt5/blob/master/examples/widgets/imageviewer.py)*. In the example below, minimal code for viewing an image has been extracted from *imageviewer.py.* The highlighted lines in black were added using the Qt5 *imagegesture* example *(http://doc.qt.io/qt-5/qtwidgets-gestures-imagegestures-example.html)* to add the pinch gesture to the following example.

```
#!/usr/bin/env python
from PyQt5.QtCore import QDir, Qt, QEvent
from PyQt5.QtGui import QImage, QPixmap
from PyQt5.QtWidgets import (QApplication, QFileDialog, QLabel,
                             QmainWindow, QSizePolicy)

from PyQt5.QtWidgets import QGesture, QGestureEvent,
QPinchGesture

class ImageViewer(QMainWindow):
```